MOVIE RECOMMENDATION SYSTEM

College Mini Project Report

Editable academic report template — fill in your personal and institution details before submission.

Project Title	Movie Recommendation System
Technology	Python, Flask, Pandas, scikit-learn, HTML, CSS
Machine Learning	TF-IDF Vectorization and Cosine Similarity
Prepared by	Name: _____
Course / Department	_____
Institution	_____
Project Guide	_____
Academic Year	_____

ABSTRACT

The Movie Recommendation System is a web-based application that helps users discover films similar to a movie they already like. The system uses content-based filtering by comparing movie genres, plot summaries, and keywords. Text features are transformed into TF-IDF vectors, and cosine similarity is used to rank movies by textual similarity. The application is implemented with Python and Flask, with a simple HTML/CSS interface. A small sample dataset is included so the starter project can run without an external database. The system is intended as an educational prototype and can be expanded with a larger, properly licensed dataset.

Keywords: Movie recommendation, content-based filtering, TF-IDF, cosine similarity, Python, Flask.

TABLE OF CONTENTS

- 1. Introduction
- 2. Problem Statement
- 3. Objectives
- 4. Existing and Proposed Systems
- 5. System Requirements
- 6. System Design and Architecture
- 7. Methodology and Algorithm
- 8. Modules
- 9. Implementation
- 10. Testing
- 11. Advantages and Limitations
- 12. Future Enhancements
- 13. Conclusion
- 14. References

1. INTRODUCTION

Online streaming platforms provide large movie libraries. Finding a film that matches a user's interests can be difficult when there are many choices. A recommendation system narrows the choices by identifying items similar to a selected movie. This project demonstrates a content-based recommendation approach using movie metadata.

2. PROBLEM STATEMENT

Users may spend significant time searching for movies that fit their interests. The aim is to develop a simple application that accepts a selected movie and recommends other movies with similar genres, descriptions, or keywords.

3. OBJECTIVES

- Build a simple, user-friendly movie recommendation website.
- Represent movie descriptions as numerical text features.
- Compare movies using cosine similarity.
- Display a ranked list of similar titles with genres, year, and overview.
- Provide a starter application that can be extended with more data.

4. EXISTING AND PROPOSED SYSTEMS

Existing system: Manual browsing or searching by title can be time-consuming and may not reveal similar films.

Proposed system: The user selects a movie from a dropdown. The application compares its content features with other movies and presents the top five similar results.

5. SYSTEM REQUIREMENTS

Hardware: Computer or laptop, 4 GB RAM recommended, and enough disk space for Python and project files.

Software: Python 3.9 or newer, pip, a web browser, and a code editor such as VS Code.

Libraries: Flask, pandas, and scikit-learn.

6. SYSTEM DESIGN AND ARCHITECTURE

The system follows a simple web application architecture:

User
↓ selects a movie
Flask Web Interface
↓ sends selected title
Recommendation Engine TF-IDF + Cosine Similarity

↓ ranks matching movies
Movie Dataset (movies.csv)
↓ returns recommendation list
Results displayed in browser

7. METHODOLOGY AND ALGORITHM

Step 1 — Data preparation: Read the CSV dataset containing title, genre, year, overview, and keywords.

Step 2 — Feature combination: Join genres, overview, and keywords into one text field for each movie.

Step 3 — TF-IDF: Convert the text into numerical vectors, giving more weight to terms that are useful for distinguishing movies.

Step 4 — Similarity calculation: Use cosine similarity to compare the feature vectors.

Step 5 — Ranking: Exclude the selected movie, sort the remaining movies by similarity, and return the top five.

Cosine similarity can be expressed as:

similarity(A, B) = (A · B) / (||A|| × ||B||)

The displayed percentage is a relative text-similarity score, not a movie rating or a guarantee that a user will like the film.

8. MODULES

- **Dataset module:** Loads movie metadata from movies.csv.
- **Feature processing module:** Combines the text fields and builds TF-IDF vectors.
- **Recommendation module:** Calculates similarity and selects the closest movies.
- **Web interface module:** Provides a dropdown, submit button, and recommendation cards.
- **About module:** Explains the project's purpose and technologies.

9. IMPLEMENTATION

The main application is written in app.py. Flask handles browser requests and renders HTML templates. Pandas loads the CSV file. TfidfVectorizer converts the text features into vectors, while cosine_similarity computes pairwise similarity. The interface files are stored in the templates folder, and style.css provides the visual design.

Project folder structure:

```
movie_recommendation_system/
■■■ app.py
■■■ movies.csv
■■■ requirements.txt
■■■ README.md
■■■ templates/
■ ■■■ index.html
■ ■■■ about.html
■■■ static/
■■■ style.css
```

How to run the project:

1. Install Python 3.9 or newer.
2. Open a terminal in the extracted project folder.
3. Run: `pip install -r requirements.txt`
4. Run: `python app.py`
5. Open `http://127.0.0.1:5000` in a browser.

10. TESTING

Test case	Input / action	Expected result
TC-01	Open home page	Movie selection list appears
TC-02	Select a valid title	Similar movie cards appear
TC-03	Submit without a selection	Browser asks for a movie selection
TC-04	Use a title not in the dataset	Not-found message appears
TC-05	Open About Project	Project overview is displayed

Testing note: Run these cases on your own computer and update the report with actual results and screenshots. Do not claim a test passed unless you have verified it.

11. ADVANTAGES AND LIMITATIONS

Advantages: Simple interface, fast recommendations for a small dataset, explainable content-based approach, and no user account required.

Limitations: The included dataset is small and illustrative. Recommendations depend on metadata quality and cannot learn personal taste from user behaviour. Similarity values are not ratings.

12. FUTURE ENHANCEMENTS

- Use a larger movie dataset with appropriate licensing and attribution.
- Add movie posters through a permitted API.
- Add user ratings and collaborative filtering.
- Add filters for genre, year, and language.
- Deploy the application to a web hosting platform.
- Add a database and user accounts if required.

13. CONCLUSION

The Movie Recommendation System demonstrates how text-processing techniques can be used to find similar movies. By combining movie metadata, TF-IDF vectorization, and cosine similarity, the application returns ranked recommendations through a web interface. It provides a foundation for further development into a larger and more personalized recommendation platform.

14. REFERENCES

- scikit-learn documentation — Feature extraction and TF-IDF: https://scikit-learn.org/
- Flask documentation: https://flask.palletsprojects.com/
- Pandas documentation: https://pandas.pydata.org/docs/
- Python documentation: https://docs.python.org/3/

Submission reminder: Add your name, register number if required by your institution, course, guide, institution name, and academic year. Include genuine screenshots of your running application if your department requires them.